Module - 04

D D M M Y Y Y Y

1. What is Virtual function? Demonstrate Virtual function with an example.

→ Virtual function is a member function in the bare class that we expect to redefine in derived classes

```
#include<iostream>
using namespace std;
class bare{
public:
Virtual void print()
{
cout << "print bare class \n";
}
void show()
{
cout << "show bare class \n";
}
};
class derived : public class bare
{
public:
void print()
{
cout << "print derived class \n";}
void show(){
cout << "show derived class \n";}
};
```

D D M M Y Y Y Y

output:-
print derived class
show base

```
int main()
{
  base* bptr;
  derived d;
  bptr = &d;
  bptr -> print();
  bptr -> show();
  return 0;
}
```

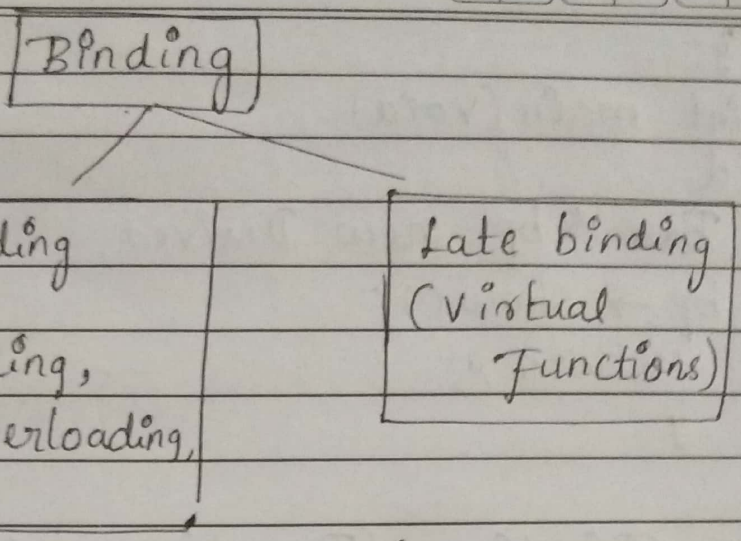
output:-
print derived class
show base class

(2) Explain 1. Early binding v/s late binding
2. pure virtual function

→ (1) Early binding v/s late binding

Binding refers to the process of converting identifiers into address.
Binding is done for each variable and functions.

Scanned with OKEN Scanner



Early Binding (compile-time polymorphism) As the name indicates compiler directly associate an address to the function call. It replaces the call with a machine language instruction that tells the mainframe to leap to the address of the function.

```
# include <iostream>
using namespace std;
class Base
{
public:
  void show(){
cout << " In Base \n"; }
};
class Derived : public Base
{
public:
void show()
{
cout << " In derived \n"; }
```

```
};
int main(void)
{
 Base *bp= new Derived;
 bp->show();
 return 0;
 }
```

Late Binding : (Run time polymorphism)
In this compiler adds code that identifies the kind of object at runtime then matches the call with right function definition.

```
# include<iostream>
using namespace std;
class Base
 {
public:
 virtual void show(){
 cout <<"In Base \n"; }
 };
class Derived : public Base
 {
public:
 Void show(){
cout <<"In Derived \n"; }
 };
```

```
int main(void)
{
Base *bp= new Derived;
 bp -> show();
return 0;
}
```

output:
In Derived

(2) pure Virtual function

→ A pure virtual function in C++ is a virtual function for which we can have an implementation. But we must override that function in the derived class.

A pure virtual function is a virtual function that has no definition within the base class.

Virtual type func_name (parameter-list) = 0;

Example :-

```
Class Test {
public:
  Virtual void show() = 0;
};
```

Scanned with OKEN Scanner

```
#include <iostream>
using namespace std;
class Base {
    int x;
public:
    virtual void func() = 0;
    int getx() {
    return x;
    }
};
class Derived : public Base {
    int y;
public:
void func() {
cout << " fun() called ";}
};
int main(void)
{
Derived d;
d.func();
return 0;
}
```

(3) What is Generic functions explain the Generic function that swap two variables with it which it is called?

→ Generics is the idea to allow type to be parameter to methods, classes and interfaces

Explanation:-

- Generic functions use type parameters like <T>, <t> etc.
- The

Syntax

- Avoid code duplication
- Same logic works for int, float, char, etc
- Improves reusability and readability

Generic function to swap two variables

- we use the keyword parameter.
- T is a type parameter.

```
#include<iostream>
using namespace std;

template<class T>
Void SwapValues(T& a, T& b)
{
  T temp;
   temp=a;
    a=b;
    b=temp;
}
int main()
{
int x=10, y=20;
float p=1.5, q=2.5;
 SwapValues(x,y);
SwapValues(p,q);
```

Scanned with OKEN Scanner

D D M M Y Y Y Y

```
Cout << " After Swapping integers : x = " << x
     << ", y = " << y << endl;
Cout << " After Swapping floats : p = " << p <<
     " , q = " << q << endl;
return 0;
}
```

(4) Write a note on (1) Generic classes
(2) Type name and export

→ (1) Generic classes

We can also use templates with the class to make it compatible with more than one data type

- In C++ programming, you might have used the vector to create a dynamic array, with every data types you pass inside the < >, ex - vector <int>.

Syntax :-

```
template <class T>
class className {

};
```

Scanned with OKEN Scanner

```
# include <iostream>
using namespace std;
  template <class T>
   class container {
   private:
      T data;
   public:
     static int count;
   container () {
      count ++; }
   Static void display static Variable () {
  Cout << count << end 1;
   }
  };
template < class T>
 int Container < T > :: count = 0;

 int main () {
   Container < int > obj 1;
   Container < float > obj 2;
   container < int > obj 3;
   Container < int > :: display Static Variable ();
   Container < float > :: display static Variable ();

  return 0;
  }
```

Output:

2

1

D D M M Y Y Y Y

① The type name and export keywords

- To we should first write the keyword template which tells the compiler that the current function or class is a blueprint
- After writing the template, we mention the keyword typename and a placeholder name (T) for a data type used by the temp function.

Class Templates

Just like the function templates in C++, we can also use class templates to create a single class that can work with the various data types.

Syntax :-

```
template < class TY class class_name
{

}
```

Function template

- function templates are similar to normal functions. As ~~to~~ Normal functions work with only one data type, but a functions template code can work on multiple data types.

Scanned with OKEN Scanner

D D M M Y Y Y Y

Syntax

```
template <class T> T function-name (Targs)
{

}
```

what is polymorphism in C++

→ Runtime polymorphism is achieved only through a pointer of the base class type.

* Also, a base class pointer can point to the objects of the base class.
* The base class pointer 'bptr' contains the address of object 'd' of the derived class.

```
#include <iostream>
Using namespace std;
class base{
public:
virtual void print(){
Cout << "print base class \n"; }
void show(){
Cout << "show base class \n"
};
class Derived{
public base
public:
void print(){
```

Scanned with OKEN Scanner

```
cout << "print derived class \n"; }
void show(){
cout << " show derived class \n"; }
};
int main()
{
base *bptr;
derived d;
bptr = &d;

bptr -> print();
bptr -> show();
return 0;
}
```

pre output:- print derived class
show base class

(6) Explain the use of templates in c++ in detail

* It is the most powerful and widely used methods added to c++, allowing us to write generic programs
* It allow us to define as blueprint for creating generic classes and functions
* If u ist straightforward we pass the data type as a parameter, so we don't

need to write the same code for different data types.

* It removes code duplication and helps us to make generic callbacks.
* Function templates are similar to normal function templates in C++ are single generic function that can work with multiple data types.
* As template are calculated at compile time rather than run time. when templat function are large and complicated, they can slow.

Scanned with OKEN Scanner

Module - 05

(7) What is Exception Handling. Write c++ program to through in division by zero exception and catch it in catch block.

→ An exception is a problem that arise during the execution of a program. A c++ exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

* Exception provide a way to transfer control from one part of a program to another try, catch and throw

→ throw - A program throws an exception when a problem shows up. this is done using a throw keyword.

try and catch keywords.

```
try
{

}
catch ()
{

}
```

```
#include <iostream>
Using namespace std;

int main (){
try {
    int age = 20;
    if (age >= 18) {
  cout << " Access granted - you are
old enough ";
} else {
  throw (age);
}
}
catch (int myNum){
  Cout << " Age is : " << myNum;
}
return 0;
}
```

(8) Explain array out of bounce exceptions with example.

→ Array out of bounds occurs when a program tries to access an array element using an index that is outside the valid range of the array.

Scanned with OKEN Scanner

```
#include <iostream>
using namespace std;

int main(){
  int a[5] = {10, 20, 30, 40, 50};

  cout << a[2]. << endl;
  cout << a[5]. << endl;
  return 0;
  }
```

Output :- 30
Garbage value.

(9) Explain

(a) width()

It is used to set the minimum field width for the next o/p only

Syntax :- cout.width(n);

Ex :-
```
#include <iostream>
using namespace std;
int main(){
cout.width(10);
cout << 123 << endl;
cout << 123 << endl;
return 0;
}
```

Scanned with OKEN Scanner

o/p :- 12 3
1 2 3

(b) precision function

It is used to set the number of digits displayed for floating point number.

Syntax :- cout. precision(n);

Ex :-
```
# include <iostream>
using namespace std;
int main(){
    float x = 12.34567;
    cout. precision(4);
    cout << x;
    return 0;
    }
```

o/p :- 12.35

(c) setf function →

It is used to set format flags for o/p

Syntax :- cout. setf (flag);

Ex :-
```
# include <iostream>
using namespace std;
int main(){
    float x = 12.3;
```

```
cout.setf(ios::showpoint);
cout << x << endl;

cout.setf(ios::fixed);
cout.precision(2);

    cout << x;
    return 0;
    }
```

o/p :- 12.3000
12.30

4. unsetf() → used to remove previously set format flags

Syntax :- cout.unsetf(flag);

Ex :-
```
# include <iostream>
using namespace std;
int main(){
float x=12.3456;
cout.setf(ios::fixed);
cout.precision(2);
cout << x << endl;

cout.unsetf(ios::fixed);
cout << x;
return 0;
}
```

Scanned with OKEN Scanner

D D M M Y Y Y Y

O/p :- 12.35
12.3456

(10) Write a C++ function to write and read time in/from binary file using fstream

→

* fstream is used for file handling
* Binary mode stores data exactly as it is in memory.
* write() → write data to file
* read() → reade data from file.

Write

```
#include <iostream>
Using namespace std;
Void Write time (){
        Time t;
ofstream.fout ("time.data", ios::binary);
Cout << "Enter hour minute & second:
fout.write ((Char*) & t, size of (t));
fout, close ();
}
```

Scanned with OKEN Scanner

Function to Read

```
#include <iostream>
Using namespace std;
Void main
    Void readTime () {
            Time, t;
    ifstream fin ("time.dat");
    fin.read ((char *) & t. size of (t));
    fin.close();
    }
```

o/p :- Enter hour minute second : 10 30 45
Time read from file : 10 : 30 : 45

Scanned with OKEN Scanner